

in RISC-V RV32I, **fixed length** (32 bits – one word)
instructions, with **four core formats**

R-type (e.g., used for “sub t0, t1, t2”)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
funct7								rs2						rs1								funct3				rd						opcode						
(7 bits)								(5 bits)						(5 bits)					(3 bits)				(5 bits)						(7 bits)									

I-type (e.g., used for “addi s0, s1, 42”)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Imm[11:0]												rs1								funct3				rd						opcode						
(12 bit immediate)																																				

S-type (e.g., used for “sw s0, 16(s1)”)

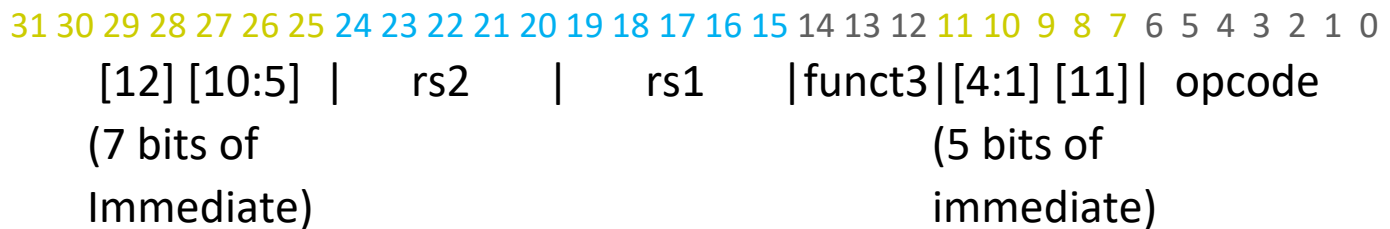
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
imm[11:5]								rs2						rs1								funct3			imm[4:0]						opcode						
(bits 5 to 11 of 12 bit immediate)																							(bits 0 to 4 of 12 bit immediate)														

U-type (e.g., used for “lui s0, 1”)

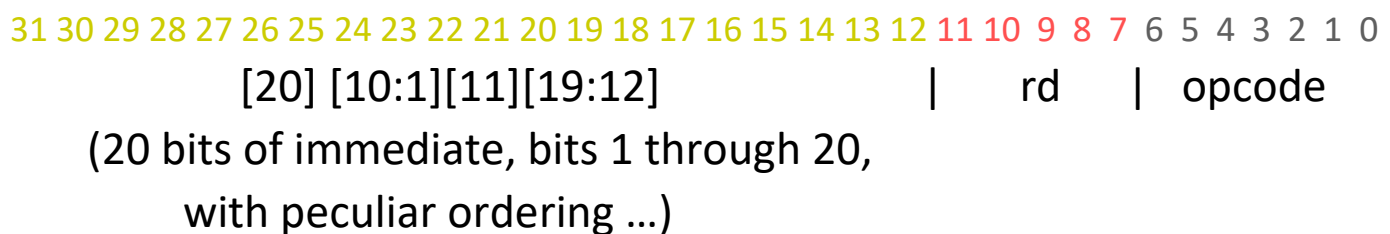
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Imm[31:12]													rd											opcode							
(high order 20 bits of 32 bit immediate)																															

and two variants of the S-type and U-type formats

B-type (variant of S-type; e.g., used for “beq s0, s1, done”)



J-type (variant of U-type; used only for the jal instruction)



some significant design features

- the **opcode** field uniquely identifies the operation only for U,J-type instructions; for the other instruction types, determines the instruction format, but also need the **funct** field(s) to identify the particular operation (*why not just have a longer opcode field?*)
- **5 bits** for each register number (*why do you think RISC-V has 32 registers rather than, say, 128?*)
- the numbers of the **register operands** always occur in the same places (*can read in parallel with decoding!*)
- no space for full-sized (32-bit) immediate operands (obviously, since an entire instruction is just 32 bits!), so dealing with immediates can get a bit messy ...

examples of RISC-V machine language instructions

`sub t0, t1, t2` $\Rightarrow$ 0100000 00111 00110 000 00101 0110011
(or `sub x5, x6, x7`)

`addi s0, s1, -4` $\Rightarrow$ 111111111100 01001 000 01000 0010011
(or `addi x8, x9, -4`) ... note that 12-bit immediate is **sign-extended** to 32 bits before adding to contents of s1

`lw s3, 8(s4)` $\Rightarrow$ 000000001000 10100 010 10011 0000011
(or `lw x19, 8(x20)`)

`sw s3, 8(s4)` $\Rightarrow$ 000000010011 10100 010 01000 0100011
(or `sw x19, 8(x20)`)

how about “`addi s0, s1, 5000?`” ... *illegal* since **constant is too big to fit in 12 bit field!** ... can instead use a sequence of three instructions, starting with **lui** to fill in the top 20 bits of s0

`lui s0, 1` $\Rightarrow$ 00000000000000000001 01000 0110111
(or `lui x8, 1`) ... note that 5000 is 0x**00001388**

can use `addi` to fill in the bottom 12 bits: “`addi s0, s0, 0x388`”
(but because of sign-extension, tricky case if leftmost of the 12 bits needed to be set to 1 ...) and then finally “`add s0, s0, s1`”

when the assembler encounters an instruction such as “la s0, A”, it uses the **auipc** instruction, an instruction that, like “lui”, uses the U-type format

auipc supports **position-independent code** – rather than having to know a fixed memory space location for “A” (in the “la s0, A” example), instead just need to know the **offset** to “A” from the address of the auipc instruction

when the auipc instruction is executed, a 32-bit value is constructed from the 20 bit immediate, with 12 zero bits added at the right, **and then added to the PC**; an “addi” can then be used after the auipc instruction, to construct the final address

auipc s0, 1 ⇒ 0000000000000000000001 01000 0010111

branches also address relative to the PC, but **using only a 13-bit offset** (**sign-extended** to 32 bits); 12 of the bits come from the instruction, a 0 bit is added at the right (so offset must be a multiple of two ... *why is this constraint reasonable?*)

beq t1, t2, skip ⇒ 0000000 00111 00110 000 01000 1100011

add t0, t1, t2

skip: add s1, s2, s3

in this case, offset should 8 bytes, or 4 half-words ... underlined part of the immediate forms bits 1 to 4 of offset (B-type format)

trythis: add t1, t2, zero

beq t1, t2, trythis ⇒ 1111111 00111 00110 000 11101 1100011